

CSS Flexbox

The 1-Dimensional Layout System

explained from first principles

A complete, corrected teaching handout: the mental model, the two axes, every property with real-world intuition, code you can paste, diagrams, and common mistakes to avoid.

In this session

1. What Flexbox is — and the one-dimensional idea
2. Container vs. items — who controls what
3. Main axis & cross axis (the whole game)
4. Container properties: direction, justify, align, wrap, gap
5. Item properties: grow, shrink, basis, align-self, order
6. The flex shorthand every dev actually writes
7. Real layouts: navbar, card grid, centering, sticky footer
8. Common mistakes + a practice game
9. Mini-project: build a Profile Card, step by step

THE BIG PICTURE

What is Flexbox, really?

Before Flexbox, we laid out pages with floats, tables, and inline-block hacks — vertical centering alone was a famous pain. Flexbox (Flexible Box Layout) was built to solve one job extremely well: distributing space and aligning items along a single line.

First principle

Flexbox is a ONE-DIMENSIONAL layout system. It arranges items in a single direction at a time — either a row (horizontal) or a column (vertical). If you need to control rows AND columns together as a true grid, that is CSS Grid's job. Flexbox = one line; Grid = both.

The mental model: you have a box (the container). You tell that box a direction. The box then lines its children up along that direction and decides how leftover space is shared and how items align. You mostly configure the container; the items just follow — unless an item speaks up for itself.

Parent & child: who is in charge



The container controls arrangement; direct children become flex items.

- The parent you put `display: flex` on is the flex container.
- Its DIRECT children become flex items automatically. Grandchildren are NOT flex items (unless their own parent is also a flex container).
- The container decides how its children are arranged — direction, spacing, alignment. This is the line 'Parent decides how the children inside it are arranged.'

Real-world analogy

Think of a shelf (container) and books (items). You, arranging the shelf, decide: do books stand left-to-right or stack top-to-bottom? Are they pushed to the left, centered, or spread out? Each book can still be told to sit differently, but the shelf sets the default rule.

A quick note your original notes got right

A `<div>` is a block element: by default it takes the full available width of its parent, and its height grows to fit its content. The moment you set `display: flex` on it, its children stop stacking vertically and start flowing along the main axis instead.

The one line that starts everything

```
/* turn any element into a flex container */
.container {
  display: flex; /* items now flow along the main axis */
}
```

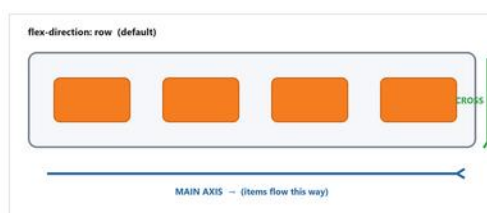
THE CORE IDEA — GET THIS AND EVERYTHING ELSE IS EASY

Main axis & cross axis

Every flex container has two axes. Almost every Flexbox property is really just 'do something along the main axis' or 'do something along the cross axis'. Get these two straight and the whole system clicks.

The rule: flex-direction chooses the MAIN axis. The CROSS axis is always perpendicular to it. Whatever direction the flex flows in is the main axis; 90° to that is the cross axis.

Default: flex-direction: row



- Main axis = horizontal (left → right). Items line up in a row.
- Cross axis = vertical (top → bottom).

flex-direction: column



- Main axis = vertical (top → bottom). Items stack.
- Cross axis = horizontal (left → right).

This is the #1 source of confusion

justify-content and align-items SWAP their visual effect when you change flex-direction, because they are tied to the axes, not to 'horizontal/vertical'. In a column, justify-content now moves items up/down and align-items moves them left/right. Don't memorize 'horizontal = justify' — memorize 'main axis = justify'.

There are also the reverse variants: row-reverse and column-reverse, which flip the start and end of the main axis (items flow right→left or bottom→top).

CONTAINER PROPERTIES

Aligning along the axes

justify-content — spacing along the MAIN axis

justify-content controls how items are positioned and how free space is distributed along the main axis. It answers: 'once my items are placed, what do I do with the leftover space in the flow direction?'



flex-start pack items at the start (default).

center pack items in the middle.

flex-end pack items at the end.

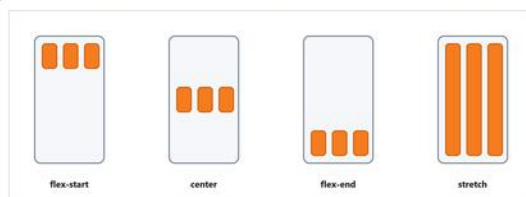
space-between first & last hug the edges; equal gaps between.

space-around equal space around each item (edges get half-gaps).

space-evenly every gap — including the edges — is exactly equal.

align-items — alignment along the CROSS axis

align-items controls how items sit on the cross axis (perpendicular to the flow). It answers: 'how tall/short should items be, and where do they sit across the line?'



stretch items stretch to fill the cross axis (default, if no fixed size).

flex-start align to the start of the cross axis.

center center on the cross axis.

flex-end align to the end of the cross axis.

baseline align items by their text baselines.

The famous perfect-center trick

display: flex; justify-content: center; align-items: center; on a container centers its child both horizontally AND vertically. This one recipe replaced a decade of vertical-centering hacks. Keep it in muscle memory.

Dead-center anything

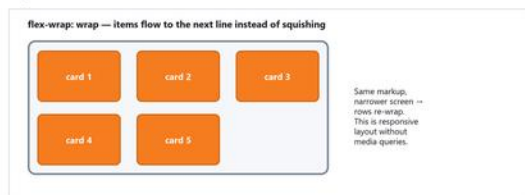
```
.hero {
  display: flex;
  justify-content: center; /* center on main axis */
  align-items: center;    /* center on cross axis */
  height: 100vh;
}
```

CONTAINER PROPERTIES

Wrapping & spacing

flex-wrap — let items flow onto new lines

By default (flex-wrap: nowrap) all items are forced onto ONE line — if there isn't enough room they shrink to fit and can overflow. flex-wrap: wrap lets items move to the next line when they run out of space. This is the backbone of responsive layouts.



Correction & nuance

Your notes said 'flex-wrap is used to make responsive websites'. That's the right instinct, but be precise: wrap lets rows re-flow as the screen narrows, which HELPS responsiveness — but real responsive design usually combines wrap with flex-basis / min-width and sometimes media queries. Wrap alone is a big part of the story, not the whole story.

gap — clean spacing between items

gap sets the space BETWEEN flex items without adding space on the outer edges. Prefer it over margins on children — it's cleaner and doesn't create awkward edge margins.

Modern spacing — no margin hacks

```
.cards {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 16px;           /* row and column gap */  
  /* gap: 12px 24px;   row-gap column-gap */  
}
```

align-content — spacing of the LINES (only when wrapped)

A subtle but important one your notes didn't cover: when items wrap onto multiple lines, align-content distributes the LINES along the cross axis (flex-start, center, space-between, stretch, ...). It only has an effect when there are multiple lines. Think of align-items as 'within a line' and align-content as 'between the lines'.

ITEM PROPERTIES — SET THESE ON THE CHILDREN

How items share space

So far we configured the container. These next properties go on the individual items and decide how each one behaves when there is extra room or not enough room.

flex-grow — sharing EXTRA space

Default flex-grow is 0, meaning items do not grow — leftover free space stays empty. Give items a positive number and they grow to absorb the free space, split in proportion to their grow values.



Get the math exactly right

flex-grow shares the FREE (leftover) space in the ratio of the grow values — it is NOT a share of the whole container. Example: 100px of free space, three items with grow 1, 2, 1 (total 4). They receive 25px, 50px, 25px ADDED to their base sizes. So an item's final size = base size + its share of free space. Your note's '1/(1+2)' was on the right track but remember it's the leftover space being divided, and it's divided by the SUM of all grow values.

```
.sidebar { flex-grow: 1; } /* takes 1 share of free space */  
.main { flex-grow: 3; } /* takes 3 shares — 3x wider growth */
```

flex-shrink — giving up space when cramped

Default flex-shrink is 1: when the container is too small, items shrink proportionally to fit. Set it to 0 and an item refuses to shrink (great for a fixed-width sidebar or an icon). A higher number means an item shrinks MORE aggressively than its siblings.

Correction

Your note said 'flex-shrink: 4 means you shrink 4 times more'. Directionally right, but the real formula weights each item by shrink-factor × its base size, so it's not a flat 4×. The practical takeaways you actually need: 0 = never shrink, 1 = default, higher = give up space first.

```
.logo { flex-shrink: 0; } /* never squish the logo */  
.search { flex-shrink: 1; } /* the search bar gives up space first */
```

flex-basis — the starting size along the MAIN axis

flex-basis sets an item's initial main-axis size BEFORE grow/shrink apply. In a row it behaves like a starting width; in a column, like a starting height. This matches your note's insight: it's not literally 'width', it's 'size along the main axis'.

Precision point

flex-basis does take priority over the width property in a row (auto is the default, meaning 'use the content/width'). But box-sizing and min-width can still influence the final result. Mental model: basis = the size the item WANTS before free space is grown or shrunk away.

ITEM PROPERTIES

The flex shorthand & per-item overrides

flex: the shorthand you'll actually type

In real code you rarely write the three properties separately. The flex shorthand combines grow, shrink, and basis in one line — and memorizing three or four patterns covers 95% of cases.

The four patterns worth memorizing

```
/* flex: <grow> <shrink> <basis> */  
.a { flex: 1; }           /* 1 1 0 → grow & shrink from 0 basis */  
.b { flex: auto; }        /* 1 1 auto → size to content, then flex */  
.c { flex: none; }        /* 0 0 auto → fixed, never grow/shrink */  
.d { flex: 0 0 240px; }    /* fixed 240px sidebar */
```

Handy tip

'flex: 1' on every child of a row is the fastest way to split a bar into equal columns. Learn this first — it's the single most-used Flexbox line in production code.

align-self — one item breaks the container's rule

align-items sets the cross-axis alignment for ALL items. align-self on a single item overrides that just for itself. This is exactly your notes' example: a container centers everything, but item 2 sets align-self: flex-start and jumps to the top while items 1, 3, 4 stay centered.



```
.container { display: flex; align-items: center; }  
.item-2 { align-self: flex-start; } /* only this one opts out */
```

order — rearrange without touching the HTML

A useful one your notes didn't mention: order (default 0) changes the VISUAL order of items without changing the HTML source order. Lower numbers come first. Handy for responsive tweaks — but use sparingly, because it can hurt keyboard/screen-reader order if overused.

PUTTING IT TOGETHER

Real-world layouts

1. A navigation bar (logo left, links right)

```
<nav class="navbar">
  <div class="logo">MySite</div>
  <ul class="links"> ... </ul>
</nav>

.navbar {
  display: flex;
  justify-content: space-between; /* logo left, links right */
  align-items: center;           /* vertically centered */
  padding: 0 24px;
}
```

2. A responsive card grid

```
.cards { display: flex; flex-wrap: wrap; gap: 20px; }
.card { flex: 1 1 280px; } /* grow, shrink, min 280px */
/* cards fill the row, then wrap when they'd get narrower than 280px */
```

3. Sticky footer (content fills, footer sticks to bottom)

```
body { display: flex; flex-direction: column; min-height: 100vh; }
main { flex: 1; } /* main grows to push footer down */
footer { /* sits at the bottom no matter how short the page */ }
```

4. Perfect centering (again — it's that common)

```
.center { display: flex; justify-content: center; align-items: center; }
```

WATCH OUT FOR THESE

Common mistakes & the recap

- Forgetting justify/align flip with direction. In a column, justify-content moves items vertically. Always ask: where is my main axis right now?
- Expecting align-content to do something with a single line. It only affects MULTIPLE wrapped lines.
- Using margins between children when gap is cleaner and edge-safe.
- Reaching for Flexbox for a true 2D grid. If you're controlling rows AND columns together, use CSS Grid.
- Over-using order and reordering visually away from the DOM — it can break tab/reading order for accessibility.
- Assuming flex-grow divides the whole width. It divides the LEFTOVER space, by the sum of grow values.

30-second recap

display: flex makes a container; direct children become items.

flex-direction picks the main axis (row = default, or column).

justify-content spacing along the MAIN axis.

align-items alignment along the CROSS axis.

flex-wrap let items flow to new lines (responsive).

gap clean space between items.

flex (grow/shrink/basis) how each item shares/gives up space.

align-self / order per-item overrides.

Practice

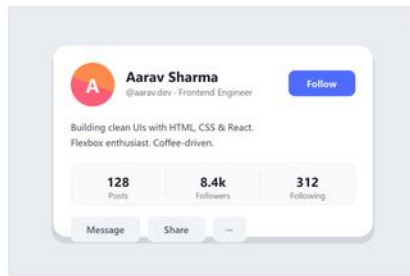
1) Positioning in CSS — revise static, relative, absolute, fixed (and sticky!). 2) Play Flexbox Froggy to build muscle memory: flexboxfroggy.com — it teaches justify-content, align-items, direction and order through a fun frog-on-lilypad game.

MINI-PROJECT — BUILD IT YOURSELF

A Profile Card with Flexbox

Let's put everything together in one small, real component: a profile card. It uses NOTHING but Flexbox for layout — no floats, no positioning, no grid. Follow it step by step; each step adds one flex container.

The finished result



This is exactly what the code below produces.

Notice the card is really FOUR flex containers nested together. Once you see the layers, the code writes itself:

How the card is 3 nested flex layers

- 1 Card = flex column (stacks the sections top→bottom)
- 2 Header = flex row (avatar | name -- | follow button)
- 3 Stats = flex row, space-around (3 equal columns)
- 4 Actions = flex row, gap (buttons in a line)

MINI-PROJECT

Step 1 — the HTML skeleton

First, plain HTML with clear class names. No styling yet — just the structure. Save this as index.html.

index.html — structure only

```
<div class="card">

  <!-- header: avatar + name + follow button -->
  <div class="header">
    <div class="avatar">A</div>
    <div class="who">
      <h2>Aarav Sharma</h2>
      <p>@aarav.dev · Frontend Engineer</p>
    </div>
    <button class="follow">Follow</button>
  </div>

  <p class="bio">Building clean UIs with HTML, CSS & React.</p>

  <!-- stats: 3 equal columns -->
  <div class="stats">
    <div><strong>128</strong><span>Posts</span></div>
    <div><strong>8.4k</strong><span>Followers</span></div>
    <div><strong>312</strong><span>Following</span></div>
  </div>

  <!-- action buttons in a row -->
  <div class="actions">
    <button>Message</button>
    <button>Share</button>
    <button>...</button>
  </div>

</div>
```

MINI-PROJECT

Step 2 — make the card a flex COLUMN

The card stacks its sections top to bottom. That is a flex container with `flex-direction: column`. `gap` adds even spacing between every section — no margins needed.

```
.card {  
  display: flex;  
  flex-direction: column; /* header, bio, stats, actions stack */  
  gap: 18px;             /* even space between sections */  
  width: 380px;  
  padding: 24px;  
  background: #fff;  
  border-radius: 18px;  
  box-shadow: 0 10px 30px rgba(0,0,0,.08);  
  font-family: system-ui, sans-serif;  
}
```

Why column here?

The card's job is to STACK sections vertically. That is a one-dimensional, top-to-bottom flow — exactly what `flex-direction: column` is for. Each section inside can then be its own row.

Step 3 — the header row (the key trick)

The header is a flex ROW: avatar on the left, name next to it, and the Follow button pushed all the way to the right. The magic is `margin-left: auto` on the button — it eats all the free space on its left and shoves itself to the end.

```
.header {  
  display: flex;  
  align-items: center; /* avatar, text & button vertically centered */  
  gap: 14px;  
}  
.follow {  
  margin-left: auto; /* ← pushes the button to the far right */  
}
```

margin-left: auto — remember this one

In a flex row, an auto margin absorbs all leftover free space on that side. `margin-left: auto` on the last item is the cleanest way to make a 'logo left, button right' bar. You'll use it constantly in navbars and toolbars.

MINI-PROJECT

Step 4 — stats in 3 equal columns

The stats strip is a flex row where the three blocks share the width evenly. `justify-content: space-around` spreads them out; each little block is itself a tiny flex column so the number sits above its label, centered.

```
.stats {
  display: flex;
  justify-content: space-around; /* 3 equal columns */
  background: #f8fafc;
  border-radius: 14px;
  padding: 14px 0;
}
.stats div {
  display: flex;
  flex-direction: column; /* number over label */
  align-items: center; /* centered horizontally */
}
```

Step 5 — the action buttons row

Finally, a simple flex row with a gap puts the buttons in a neat line. Add `flex-wrap` so they drop to a second line on very narrow screens instead of overflowing.

```
.actions {
  display: flex;
  gap: 12px;
  flex-wrap: wrap; /* wrap on tiny screens */
}
.actions button { flex: 1; } /* optional: equal-width buttons */
```

You just built a real component

Every part of this card is a flex container: the card (column), the header (row with an auto margin), the stats (row, space-around, with nested columns), and the actions (row with gap + wrap). This is exactly how production UIs are laid out. Rebuild it from memory once — that's the real learning.

Challenge: turn the header into `flex-direction: column` on narrow screens (inside a media query) so the avatar sits above the name on mobile. Then try making the whole card a horizontal row of TWO cards side by side using another flex container around them.